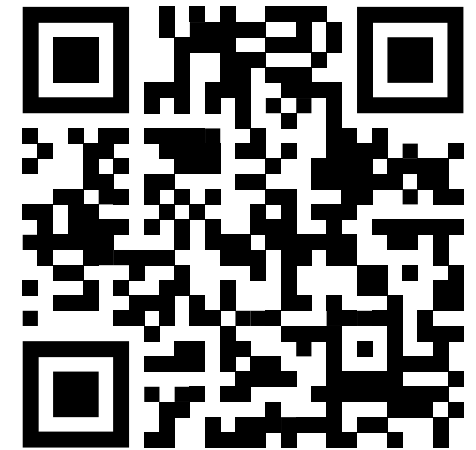


Programmierung in C++

Fehlerbehandlung

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Dan Eisenkrämer



<https://poll.hs-kempten.de/poll/>



Fehlertypen

Lernziele

- Die verschiedenen Fehlertypen und ihre Auswirkungen wurden verstanden.
 - Fehler können den verschiedenen Typen zugeordnet werden.

Fehlertypen

Übersicht



■ **Compiler-Fehler**

- Werden vom Compiler während der Kompilierung gefunden
- Unterteilen sich in Typfehler und Syntaxfehler

■ **Linker-Fehler**

- Werden vom Linker gefunden, wenn die Objekt-Dateien zu einer Executable zusammengeführt werden sollen

■ **Laufzeitfehler**

- Werden zur Laufzeit durch weitere Überprüfungen auf den tatsächlichen Instanzen entdeckt

■ **Logikfehler**

- Werden vom Entwickler oder Anwender durch unerwünschtes Verhalten (z.B. falsche Ausgaben oder Endlosschleifen) bemerkt

Fehlertypen

Beispiel



```
/// <summary>
/// Berechnet den ganzzahligen Flächeninhalt eines Rechteck
/// </summary>
/// <param name="length">Länge des Rechtecks</param>
/// <param name="width">Breite des Rechtecks</param>
/// <returns>Ganzzahliger Flächeninhalt</returns>
int area(int length, int width) { return length + width; }
```

Logikfehler:

Die Summe wird nicht das erwartete Ergebnis liefern.

```
// Berechnet den Flächeninhalt eines Rechtecks mit Gleitkommazahlen
double area(double length, double width);
```

Linker-Fehler:

Nicht aufgelöstes externes Symbol => Die Funktion ist nicht definiert.

```
// Berechnet den Flächeninhalt eines gerahmten Rechtecks
int area(int x, int y, int frameWidth) { return (x - frameWidth) * (y - frameWidth); }
```

Syntaxfehler:

Es wurde ein „;“ erwartet.

```
int main() {
```

```
    int syntax = area(7,3);
```

```
    int type = area(7);
```

```
    double link = area(5.0, 1.2);
```

```
    double runtime = area(3, 2) / area(3, 2, 2);
```

```
}
```

Typfehler:

Falsche Anzahl an Argumenten.

(möglicher) Logikfehler:

Ergebnis der ganzzahligen Division wird abgeschnitten.

Laufzeitfehler:

Integer Division durch Null.

Fehlertypen

Compiler-Fehler



- Während der Code kompiliert wird, überprüft der **Compiler** den Code auf Syntax- und Typfehler
 - Ist die erste Verteidigungslinie bei der Fehlerbehandlung
 - Compiler-Fehler geben einen guten Hinweis, wo sich Fehler verstecken, für die man ansonsten stundenlang gesucht hätte

```
int area(int length, int width) { return length * width; }
```

Syntaxfehler	Typfehler	Kein Compiler Fehler!
<pre>int s1 = area(7,3; // => ")" erwartet int s2 = area(7,3) // => ";" erwartet Int s3 = area(7,3); // => "Int" ist kein Typ</pre>	<pre>int t1 = arena(7,3); // => Nichtdeklarierte Funktion int t2 = area(7); // => Falsche Anzahl Argumente int t3 = area("seven",2); // => 1. Argument hat falschen Typ</pre>	<pre>int n1 = area(10,-7); // OK: Aber Seitenlänge -7? int n2 = area(10.7,9.3); // OK: Aber ruft area(10,9) auf char n3 = area(100,9999); // OK: Aber schneidet Ergebnis ab</pre>

Fehlertypen

Linker-Fehler



- Jedes Programm besteht aus unterschiedlichen kompilierten Teilen
 - Der **Linker** verknüpft diese Teile und wirft Fehler, die durch die Verlinkung auftreten
 - Jeder kompilierte Teil, der auf eine deklarierte Funktion zugreift, muss auf die entsprechende Definition zugreifen können.
 - Jede Funktion darf nur einmal definiert werden.
 - Es muss exakt ein Programmeinstiegspunkt (main-Funktion) existieren

```
#pragma once  
int area(int length, int width);
```

math.h

```
#include "math.h"  
int area(int length, int width) {  
    return length * width;  
}
```

math.cpp

```
#include "math.h"  
int area(int length, int width) {  
    return length * width;  
}  
  
int main() {  
    int link = area(7, 3);  
}
```

main.cpp

Linkerfehler:
Mindestens ein mehrfach
definiertes Symbol
gefunden.

Fehlertypen

Laufzeitfehler



- Wenn das Programm ohne Compiler- und Linker-Fehler startet, fängt der „Spaß“ meist erst an.
 - Werden **zur Laufzeit** durch weitere Überprüfungen auf den tatsächlichen Instanzen entdeckt
 - Führen dazu, dass der **Code nicht ausgeführt** werden kann
 - Programm stürzt ab und häufig erhält man eine kryptische Fehlermeldung

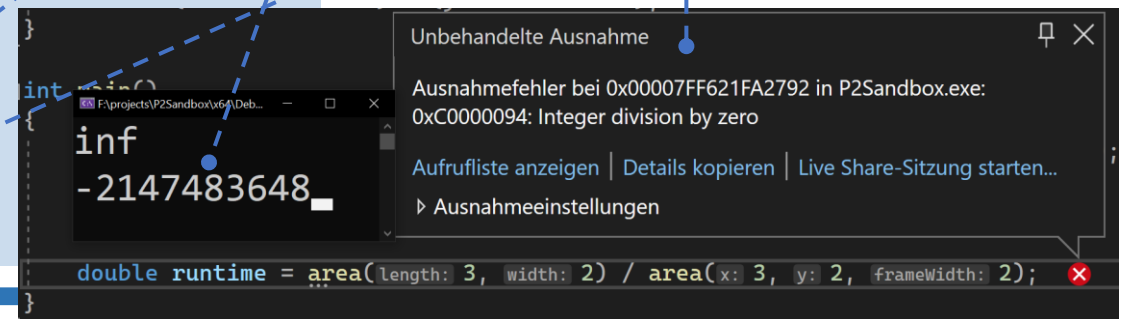
```
// Berechnet den ganzzahligen Flächeninhalt eines Rechteck
int area(int length, int width) { return length * width; }
// Berechnet den Flächeninhalt eines gerahmten Rechtecks
int area(int x, int y, int frameWidth) { return area(x-frameWidth,y-frameWidth); }

int main() {
    double ratio = double(area(3, 2)) / area(3, 2, 2); // OK?
    std::cout << ratio << '\n'; // OK?
    std::cout << int(ratio); // OK?

    double runtime = area(3, 2) / area(3, 2, 2); // Laufzeitfehler
}
```

(möglicherweise) **Kein Fehler!**:
Das Programm kann irgendwie noch weitermachen. Double-Division liefert Inf (NaN) und läuft ggf. trotzdem weiter.

Laufzeitfehler:
Integer division by zero



Fehlertypen

Logikfehler



- Das Programm läuft und stürzt nicht ab, aber es verhält sich **nicht wie geplant und erfordert**
 - Dann handelt es sich um Logikfehler, die erst einmal gefunden werden müssen
 - Nur anhand der **Fehlerwirkung** (also das, weshalb man sagt „Moment, da stimmt was nicht“) lässt sich nur schwierig auf die Fehlerursache schließen
- Mögliche Lösungsansätze:
 - Break-Points setzen und während der Laufzeit die Werte überprüfen
 - Debug-Ausgaben erzeugen (auf der Konsole oder in Log-Dateien)
 - Testfälle implementieren (*mehr dazu im Wahlpflichtmodul Programmieren 3*)

```
int area(int length, int width) { return length + width; }
int area(int x, int y, int frameWidth) { return area(x - frameWidth, y - frameWidth); }
int main() {
    double a = area(3, 1);
    double ratio = a / area(3, 1, 2);
    std::cout << uint64_t(ratio);
}
```

Ausgabe:

9223372036854775808

Logikfehler:

1. Für die Fläche werden die Werte addiert.
 2. Die Funktion akzeptiert negative Werte.
- => Durch die Division durch Null entsteht ein Wert, der den Fehlern kaum noch zuordbar ist.

Fehlertypen

Zusammenfassung



■ Compiler-Fehler

- Werden vom Compiler während der Kompilierung gefunden
- Können in weitere Fehlertypen unterteilt werden:
 - Syntaxfehler
 - (Daten-)Typfehler

■ Linker-Fehler

- Werden vom Linker gefunden, wenn die Objekt-Dateien zu einer Executable zusammengeführt werden sollen

■ Laufzeitfehler

- Werden zur Laufzeit durch weitere Überprüfungen auf den tatsächlichen Instanzen entdeckt
- Können in weitere Fehlertypen unterteilt werden:
 - Fehler, die vom Computer entdeckt werden (Hardware- & Betriebssystem-Fehler)
 - Fehler, die in einer Bibliothek geworfen werden (z.B. der Standardbibliothek)
 - Fehler, die vom selbst implementierten Code entdeckt werden

■ Logikfehler

- Werden vom Entwickler oder Anwender durch unerwünschtes Verhalten (z.B. falsche Ausgaben oder Endlosschleifen) bemerkt

Fehlertypen

Verständnisfragen

PIN: 849822

<https://poll.hs-kempten.de/poll/>



- **Welche Art von Fehler werden durch den Compiler erkannt?**
 - Verweis auf ein nicht aufgelöstes, externes Symbol.
 - Zugriff auf ungültigen Speicher.
 - Fehlendes Semikolon am Ende einer Zeile.
 - Switch-Case Anweisung über einen std::string.
- **Wo/Wann ist es am leichtesten von einem Fehler auf die Ursache zu schließen?**
 - Compiler
 - Editor
 - Laufzeit
 - Linker
- **Welche Fehlertypen werden vor dem Start des Programms überprüft?**
 - Logikfehler
 - Syntaxfehler
 - Hardwarefehler
 - Linkerfehler



Exceptions

Lernziele

- Möglichkeiten, um Fehler mitzuteilen sind mit ihren Vor- und Nachteilen bekannt.
- Dem eigenen Code können Exceptions hinzugefügt werden.
 - Es ist klar, wie Fehlerobjekte geworfen und gefangen werden können.
 - Fehlerobjekte können, wenn nötig, weitergegeben werden.
 - Es wurde verstanden, wann Exceptions verwendet werden sollten und wann nicht

Exceptions



Motivation – Fehler mitteilen

- Wie können wir Fehler vorbeugen und mit Fehlern umgehen?
 - **Möglichkeit 1:** Potentielle Fehler **vor** dem Funktionsaufruf abfangen

```
int main() {  
    int x = 3;  
    int y = 1;  
    if (x < 0 || y < 0) { // Fehler abfangen  
        std::cout << "ERROR: negative length!";  
        return -1;  
    }  
    int area1 = area(x, y); // Funktionsaufruf  
    std::cout << "Area 1: " << area1 << '\n'; Area 1: 3  
    int frameWidth = 2;  
    if (x - frameWidth < 0 || y - frameWidth < 0) {  
        std::cout << "ERROR: length is smaller than frame width!";  
        return -1;  
    } ERROR: length is smaller than frame width!  
    int area2 = area(x, y, frameWidth); // Funktionsaufruf  
    std::cout << "Area 2: " << area2 << '\n';  
}
```

F:\projects\P2Sandbox\x64\Debug\P2Sandbox.exe (Prozess "8476") wurde mit Code "-1" beendet.

- Der Code wird dadurch sehr unübersichtlich
- Die Abfragen müssten vor jedem Aufruf geschrieben werden
- Dem Aufrufer der Funktion müsste die Funktionsweise genau bekannt sein, um mögliche Fehler abzufangen

Exceptions



Motivation – Fehler mitteilen

- Wie können wir Fehler vorbeugen und mit Fehlern umgehen?
 - Möglichkeit 1: Potentielle Fehler vor dem Funktionsaufruf abfangen
 - **Möglichkeit 2:** Potentielle Fehler **in** der Funktion abfangen

```
int area(int length, int width) {  
    if (length < 0 || width < 0)  
    {  
        std::cout  
            << "ERROR: negative length in area()!";  
        return -1;  
    }  
    return length * width;  
}  
  
int area(int x, int y, int frameWidth) {  
    return area(x - frameWidth, y -  
        frameWidth);  
}
```

- Es muss nur noch in der ersten *area()*-Funktion auf Fehler getestet werden
- Der Aufrufer der Funktion muss nicht wissen, welche Fehler alle überprüft werden müssen
- Wir fügen der Funktion jedoch eine Abfrage hinzu, die bei jedem Aufruf durchgeführt wird, selbst, wenn der Nutzer einen sicheren Programmablauf gewährleistet. => Performance-Nachlass
- Die Funktion kann möglicherweise nicht genug Informationen besitzen, um eine sinnvolle Fehlermeldung auszugeben.
 - Hier: Wenn die Länge kleiner als die Rahmendicke ist.

Exceptions



Motivation – Fehler mitteilen

- Wie können wir Fehler vorbeugen und mit Fehlern umgehen?
 - Möglichkeit 1: Potentielle Fehler vor dem Funktionsaufruf abfangen
 - **Möglichkeit 2:** Potentielle Fehler **in** der Funktion abfangen

```
int area(int length, int width) { /* ... */ }
int area(int x, int y, int frameWidth) { /* ... */ }

int main()
{
    int area1 = area(3, 1);
    if (area1 < 0) return area1;
    std::cout << "Area 1: " << area1 << '\n';

    int area2 = area(3, 1, 2);
    if (area2 < 0) return area2;
    std::cout << "Area 2: " << area2 << '\n';
}
```

- Der von *area()* zurückgegebene Fehlerwert muss trotzdem vom Aufrufer abgefangen werden
 - Es sind doch wieder mehrere Checks notwendig
 - Vergisst der Aufrufer das Abfangen des Wertes kann dies unvorhersehbare Folgen haben
- Viele Funktionen haben nicht die Möglichkeit im Datentyp Fehlerwerte zu inkludieren
 - Wenn z.B. auch negative valide Rückgabewerte sind

Exceptions



Motivation – Fehler mitteilen

- **Möglichkeit 1: Potentielle Fehler vor dem Funktionsaufruf abfangen**
 - Hauptvorteil: Es können spezifischere und eindeutigere Fehlermeldungen erzeugt werden.
 - Hauptproblem: Notwendige Überprüfungen müssen bekannt und durchgeführt sein.
- **Möglichkeit 2: Potentielle Fehler in der Funktion abfangen**
 - Hauptvorteil: Der Aufrufer muss nicht wissen, worauf alles Fehler geprüft werden müssen.
 - Hauptproblem: Im Rückgabebetyp kann nicht immer ein Fehlerwert inkludiert werden.
- **Möglichkeit 3: Exceptions**
 - In der Funktion wird auf mögliche Fehler überprüft und ein Fehlerobjekt „geworfen“
 - Der Aufrufer kann das Fehlerobjekt „fangen“, um spezifischere Fehlermeldungen auszugeben und den fehlerhaften Programmablauf geregelt zu beenden
 - Exceptions werden für den Ausnahmefall implementiert und dienen nicht dem regulären Programmablauf



Exceptions

Bad Arguments



- Durch **throw** kann eine Funktion mit einem Fehlerobjekt verlassen werden, welches nicht mit dem Rückgabewert übereinstimmen muss
 - Dazu kann z.B. eine leere Klasse erstellt werden
 - Mit **catch** kann dieses Fehlerobjekt aufgefangen werden
- Trennung zwischen Fehlerüberprüfung, Fehlererkennung und Fehlerbehandlung
 - Der **try**-Block gibt an, wo Fehler gefangen werden sollen
 - Es ist nicht relevant, welcher Funktionsaufruf den Fehler geworfen hat
 - Der Funktion ist egal, wie der Fehler aufgefangen wird
 - Die Fehlerbehandlung ist getrennt vom fehlerfreien Fall

```
// Type for reporting errors in area
class BadArea {};

int area(int length, int width) {
    if (length < 0 || width < 0)
        throw BadArea();
    return length * width;
}

int area(int x, int y, int frameWidth) { /* ... */ }

int main()
try {
    int x = 1, y = 2, z = 4;
    int area1 = area(x, y);
    int area2 = area(1, z, 2);
    int area3 = area(y, z, 2);
    double ratio = area1 / area3;
}
catch (BadArea) {
    std::cout << "bad arguments to area()\n";
}
```


Exceptions

Range errors

- Häufig entstehen Fehler durch den versuchten Zugriff auf Elemente, die nicht Teil eines Containers sind
 - Der Programmierer kann gewährleisten, dass es zu keinem Range Fehler kommt
 - Aber jeder Programmierer macht Fehler, vor allem wenn er leichtsinnig wird
 - **Wo ist rechts der Fehler?**



```
int main()
{
    std::vector<int> v; // Int-Vector
    for (int x; std::cin>>x; )
        v.push_back(x); // Lies Werte ein
    // Gib Werte aus
    for (int i = 0; i<=v.size(); ++i)
        std::cout << i << ": " << v.at(i) << '\n';
    return 0;
}
```

Exceptions

Range errors



- Häufig entstehen Fehler durch den versuchten Zugriff auf Elemente, die nicht Teil eines Containers sind
 - Der Programmierer kann gewährleisten, dass es zu keinem Range Fehler kommt
 - Aber jeder Programmierer macht Fehler, vor allem wenn er leichtsinnig wird
 - **Wo ist rechts der Fehler?**
 - Falsch: `<=`
 - Richtig: `<`

```
int main()
{
    std::vector<int> v; // Int-Vector
    for (int x; std::cin>>x; )
        v.push_back(x); // Lies Werte ein
    // Gib Werte aus
    for (int i = 0; i<=v.size(); ++i)
        std::cout << i << ": " << v.at(i) << '\n';
    return 0;
}
```

Exceptions

Range errors

- Häufig entstehen Fehler durch den versuchten Zugriff auf Elemente, die nicht Teil eines Containers sind
 - Der Programmierer kann gewährleisten, dass es zu keinem Range Fehler kommt
 - Besonders während der Entwicklung darf man sich als Programmierer nicht immer trauen
- Viele Bibliotheksfunktionen werfen Fehler, die aufgefangen werden können
 - Diese können explizit, getrennt oder gebündelt mit „...“ gefangen werden
 - ***std::cerr*** funktioniert ähnlich wie *std::cout*, aber ist für Fehlermeldungen gemacht

```
0: 4
1: 5
2: -2
3: 3
Oops! Range error
```



```
#include <exception>
```

```
int main()
try {
    std::vector<int> v; // Int-Vector
    for (int x; std::cin>>x; )
        v.push_back(x); // Lies Werte ein
    // Gib Werte aus
    for (int i = 0; i<=v.size(); ++i)
        std::cout << i << ": " << v.at(i) << '\n';
    return 0;
} catch (std::out_of_range) {
    std::cerr << "Oops! Range error\n";
    return 1;
} catch (...) { // Fange alle anderen Fehler
    std::cerr <<
        "Exception: something went wrong\n";
    return 2;
}
```

Exceptions

Fehlerobjekte



```
#include <exception>
```

- Die STL stellt einige Exceptions zur Verfügung und wirft diese auch in ihren eigenen Funktionen
 - Viele Fehlerobjekte besitzen ein übergeordnetes, allgemeineres Fehlerobjekt und erben von diesem
 - Die von **runtime_error** erbenden Fehlerobjekte besitzen einen String mit zusätzlichen Informationen zu dem Fehler
 - Auf Fehlerinformationen von Objekten der Überklasse **exception** wird mit `.what()` zugegriffen

Auszug der STL
Exceptions:

logic_error

- invalid_argument
- domain_error
- length_error
- out_of_range
- future_error

runtime_error

- range_error
- overflow_error
- underflow_error
- regex_error
- system_error
 - ios_base::failure
 - filesystem::filesystem_error

- bad_typeid
- bad_cast
 - bad_any_cast
- bad_weak_ptr
- bad_alloc
- bad_exception
- ...

```
try {  
    // ...  
    return 0;  
}  
catch (std::exception& e) {  
    std::cerr << "error: " << e.what() << "\n";  
    return 1;  
}  
catch (...) {  
    std::cerr << "Oops: unknown exception!\n";  
    return 2;  
}
```

Exceptions

Fehlerobjekte

- C++ lässt es dem Programmierer nahezu komplett offen, was er werfen möchte
 - Die meisten anderen Programmiersprachen sind bei Exceptions sehr restriktiv
- Idealerweise erben eigene Fehlerobjekte von den STL Exceptions
 - Dadurch werden die eigenen Fehlerobjekte auch dann gefangen, wenn Nutzer nur `std::exception` fangen
 - Die Hierarchie der STL-Fehlerobjekte wird ausgenutzt und gibt den Nutzern zusätzliche Informationen

Dieser Catch-Block wird aufgerufen und das Programm mit Code 1 beendet

error: MyException



```
#include <exception>
```

```
class MyException : public std::runtime_error {  
public:  
    MyException()  
        : std::runtime_error("MyException") {}  
};  
  
void f() {  
    throw MyException();  
}  
  
int main()  
try {  
    f();  
    return 0;  
} catch (std::exception& e) {  
    std::cerr << "error: " << e.what() << '\n';  
    return 1;  
} catch (...) {  
    std::cerr << "Oops: unknown exception!\n";  
    return 2;  
}
```

Exceptions

Fehlerobjekte



- Fehlerobjekte können auf die unterschiedlichen Arten gefangen werden
 - Es besteht dieselbe Flexibilität wie bei Funktionsparametern und dieselbe Kompatibilität
- „**Catch by Value**“
 - Wie üblich erzeugt „Catch by Value“ **Kopien** der Fehlerobjekte
 - Vermeide daher „Catch by Value“
- „**Catch by Reference**“
 - Ist generell den anderen Möglichkeiten zu bevorzugen
 - Das referenzierte Objekt bleibt für den Catch-Block gültig, obwohl es in einem anderen Funktions-Scope erstellt wurde
- „**Catch by Pointer**“
 - Nur unter sehr speziellen Umständen sollten Fehlerobjekte via Zeiger gefangen werden
 - Es muss klar sein, wer, wo und wann der Pointer gelöscht werden soll

Exceptions

Dos & Don'ts



- Exceptions dienen **ausschließlich der Fehlerbehandlung**
 - Nutze *throw* **nicht**, um Logikfehler abzufangen, bei denen z.B. eine Variable einen unerwarteten Wert hat, das Programm aber normal weiterlaufen kann.
=> Nutze dafür Pre- und Post- Conditions
 - Nutze *throw* auf **gar keinen Fall**, wenn sich Memory Leaks oder Speicherkorruption bemerkt machen, da Exceptions dann zu weiterer Korruption und Memory Leaks führen.
=> Nutze dafür RAII
 - Nutze Exceptions **nicht** für deinen Programmablauf
 - *throw* ist **keine** alternative Möglichkeit, um einen erwünschten Wert aus einer Funktion zurückzugeben
 - *throw* ist **keine** valide Variante, um eine Schleife gewollt zu verlassen
 - Exceptions **sind** eine Alternative zur Rückgabe von Fehlercodes, die mit IF-Statements abgefangen werden müssen
 - IF-Bedingungen sind deutlich fehleranfälliger
 - IF-Bedingungen erschaffen Branches im Code, die einen erhöhten Mehraufwand und Risiko mit sich bringen, wenn man die Programmabläufe testen möchte

Exceptions

Fehlerweitergabe



- Der Fehler kann in der Funktion gefangen werden, die den Fehler behandeln soll
 - Es benötigt keine Abfrage bei jeder Verschachtelungsschicht wie bei Fehlercodes, die als Return-Value zurückgegeben werden (rechts)
 - Nur die Funktion, die den Fehler erkennt und die, die ihn behandelt, interessiert der Fehler (unten)

Exception

```
void f1() {  
    try {  
        f2();  
        // Alles hat geklappt  
    } catch (some_exception& e) {  
        // Fehlerbehandlung  
    }  
}
```

```
void f2() { /*...*/ f3(); /*...*/ }  
void f3() { /*...*/ f4(); /*...*/ }  
void f4() {  
    // ...  
    if ( /* Fehlerbedingung */ )  
        throw some_exception();  
    // ...  
}
```

Fehlercodes

```
int f1() {  
    int rc = f2();  
    if (rc == 0) {  
        // Alles hat geklappt  
    }  
    else {  
        // Fehlerbehandlung  
    }  
}  
int f2() {  
    // ...  
    int rc = f3();  
    if (rc != 0) return rc;  
    // ...  
    return 0;  
}  
int f3() {  
    // ...  
    int rc = f4();  
    if (rc != 0) return rc;  
    // ...  
    return 0;  
}  
int f4() {  
    // ...  
    if ( /* Fehlerbedingung */ )  
        return FEHLERCODE;  
    // ...  
    return 0;  
}
```


Exceptions

Re-Throw

- Fehler können **weitergeworfen** werden
 - Dazu wird der `throw`; Befehl ohne weitere Infos verwendet
 - Nützlich, um Fehlern weitere Informationen hinzuzufügen
 - Muss bedacht eingesetzt werden, um das Codeverständnis nicht zu verringern.
- Re-Throws können als „**Exception Dispatcher**“ verwendet werden
 - Erlaubt, eine einzelne Funktion zu implementieren, die mögliche Fehler von mehreren anderen Funktionen behandelt
 - Dazu wird im Fehlerfall die Funktion aufgerufen, der Fehler erneut geworfen und dann behandelt



```
void f() {  
    try {  
        throw MyException;    /* Option 1 */  
        // throw YourException; /* Option 2 */  
    } catch (MyException& e) {  
        e.addInfo("f() failed");  
        throw; // Rethrow  
    }  
}  
// Exception Dispatcher  
void handleExceptions() {  
    try { throw; // Rethrow  
    } catch (MyException& e) {  
        // Fehlerbehandlung: MyException  
    } catch (YourException& e) {  
        // Fehlerbehandlung: YourException  
    }  
}  
void main()  
try { f(); }  
catch (...) { handleExceptions(); }
```

Hier werden alle Fehler gefangen und erst in *handleExceptions* differenziert behandelt.

Exceptions



Re-Throw

- Wird ein **erbendes Fehlerobjekt** an eine Funktion weitergegeben, die Objekte der Elternklasse aufnimmt, wird beim Re-Throw ein Objekt der Elternklasse geworfen
 - Das Kindobjekt muss **manuell** weitergeworfen werden, z.B. via einer virtuelle Funktion

```
class MyExceptionBase { };
class MyExceptionDerived : public MyExceptionBase { };
void f(MyExceptionBase& e) { // Up-Cast zu Base
    throw e;
}
void g() {
    MyExceptionDerived e;
    try {
        f(e);
    } catch (MyExceptionDerived& e) {
        // Dieser Catch-Block wird NICHT aufgerufen
    } catch (...) {
        // Dieser Catch-Block wird aufgerufen
    }
}
```

Problematisch

```
class MyExceptionBase {
public: virtual void raise(){ throw *this; }
};
class MyExceptionDerived : public MyExceptionBase {
public: void raise() override{ throw *this; }
};
void f(MyExceptionBase& e) { e.raise(); }
void g() {
    MyExceptionDerived e;
    try {
        f(e);
    } catch (MyExceptionDerived& e) { // Wird aufgerufen!
    } catch (...) { // Wird NICHT aufgerufen
    }
}
```

Lösung

Exceptions



Vorsicht bei Verwendung von Exceptions

- Der Code sollte **nie** mit Try-Catch-Blöcken überfüllt werden!
Es sollte immer **gut überlegt** sein, wann man Try-Catch verwendet.
- „Ich möchte den Fehler behandeln und mit der Ausführung fortfahren, ohne weitere Fehler zu werfen. Der Aufrufer soll nicht erfahren, dass ein Fehler passiert ist, weil mit dem Fehler **korrekt umgegangen werden konnte**.“
 - Der Try-Catch-Block sollte bestehen bleiben, er ist vermutlich **gut so**
- „Die Catch-Klausel soll XYZ erledigen und den Fehler erneut werfen.“
 - **Besser:** Es könnte sich anbieten, aus dem Try-Block eine Klasse zu erstellen, deren Destruktor XYZ erledigt. (Nach dem Prinzip **RAII**)
- „Ich möchte die Fehlerinformationen extrahieren und zu einem besseren Fehler verpacken und diesen dann werfen.“
 - **Besser:** Vielleicht lässt sich die Hierarchie der Fehlerobjekte verbessern, sodass das erste Fehlerobjekt den Fehler ideal wiedergibt und kein Re-Throw notwendig ist

Exceptions



Exceptions in Konstruktor & Destruktor

- Exceptions in Konstruktoren können sinnvoll sein
 - Wenn ein Objekt nicht korrekt initialisiert/erstellt werden kann, sollte ein Fehler geworfen werden
 - Da Konstruktoren keinen Rückgabewert besitzen, können konkrete Fehler nur über Exceptions mitgeteilt werden
 - Wenn ein Konstruktor einen Fehler wirft können **keine** Memory-Leaks entstehen
 - Selbst mit `new` allokierte Objekte werden in diesem Fall sofort wieder gelöscht
 - Wenn Exceptions keine Möglichkeit sind (z.B. bei harten Echtzeitanforderungen), kann das fehlerhafte Objekt in einen „Zombie“-Zustand versetzt werden, auf den bei Aufrufen auf das Objekt immer überprüft werden muss
- Wirf **NIEMALS** Fehler in Destrukturen
 - Außer, wenn der Fehler im Destruktor selbst behandelt und behoben wird
 - Exceptions dürfen den Destruktor nicht verlassen, sonst passieren schlimme Dinge
 - Erstelle lieber ein Log-File, in dem Informationen vermerkt werden & beende das Programm

Exceptions



C++11 Noexcept

- Funktionen, die garantiert niemals Fehler werfen, können und sollten mit dem Schlüsselwort ***noexcept*** deklariert werden. `RetType function(params) noexcept;`
 - Dadurch weiß der Anwender, dass beim Aufruf der Funktion niemals Fehlerobjekte aufgefangen werden müssen.
 - Es kommt zum Programmabsturz, wenn ein Fehlerobjekt die Funktion verlässt.
 - Der Compiler kann effizientere Objektdaten generieren.
 - In einer *noexcept*-Funktion benötigt es **keinen Overhead** für *throw()* bei der Aufrufhierarchie, da sie nicht zurück traversiert werden muss, wenn ein Fehler die Funktion verlässt.
 - In diesem Fall muss nicht sichergestellt werden, dass Objekte in inverser Reihenfolge zerstört werden.
 - *noexcept* ist besonders wertvoll für Move-Operationen, Allokationen und Destruktoren.
 - Deallokations-Funktionen (Implementationen von *delete*) & Destruktoren sind **implizit *noexcept***!
 - *noexcept* sollte jedoch **nur** verwendet werden, wenn die Funktion **auch zukünftig keine Fehlerobjekte** verlassen können.
 - Die meisten Funktionen sind **exception-neutral**:
Sie werfen selbst keine Fehlerobjekte, können allerdings die geworfenen Fehlerobjekte von aufgerufenen Funktionen durchreichen bzw. weitergeben.

Exceptions



Zusammenfassung

- Exceptions werden verwendet, um **unerwarteten Programmabläufe** abzufangen und unkontrollierte **Programmabstürze** zu verhindern
 - Fehlerhafte Nutzereingaben sind zu erwarten und sollten daher **nicht** zu unerwarteten Programmabläufen führen!
- Exceptions können beliebige Objekte als **Fehlerobjekt** werfen
 - Diese Objekte können **aufgefangen** werden, um den Fehler angemessen zu behandeln
 - Fehlerobjekte sollten eine **geeignete Hierarchie** besitzen und erben idealerweise von den Exceptions der Standardbibliothek
 - Durch geworfene Fehler in **Konstruktoren** können keine Memory Leaks entstehen
 - Fehlerobjekte werden durch alle Funktionsaufrufe **weitergegeben**, bis sie aufgefangen werden, und können nach dem Auffangen erneut geworfen werden.
 - Funktionen, die nie Fehlerobjekte werfen, sollten mit **noexcept** gekennzeichnet werden
- **Try-Catch-Blöcke** machen auf kritische Stellen im Code aufmerksam und sollten nicht verschwenderisch verwendet werden.

Exceptions

Verständnisfragen

PIN: 849822

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Es ist meist geschickter, wenn eine Funktion im Fehlerfall bestimmte Werte zurückgibt anstatt ein Fehlerobjekt zu werfen.
- Exceptions sollten verwendet werden, um den Programmfluss zu beschleunigen.
- Die Verwendung von Exceptions ist sinnvoll, um das Verfolgen von Fehlern zu vereinfachen.
- Eine catch-Anweisung kann mehrere Ausnahme-Typen in C++ gleichzeitig behandeln.
- Um Exceptions werfen zu können, müssen eigene Fehlerobjekte implementiert werden.

■ Wofür nutzt man das Schlüsselwort `catch` in C++?

- Kennzeichnet den Programmcode, der ausgeführt wird, wenn ein bestimmter Fehlerfall auftritt.
- Für die sogenannte Switch-Catch-Fallunterscheidung.
- Um geworfene Objekte zu fangen.

■ Wofür nutzt man das Schlüsselwort `noexcept` in C++?

- Ermöglicht, dass der Compiler effizientere Objektdaten erzeugen kann.
- Kennzeichnet, dass in einer Methode keine try-catch-Blöcke existieren.
- Fehlerobjekte dürfen derartig gekennzeichnete Methoden niemals verlassen.



Pre- & Post- Conditions

Lernziele

- Die Begriffe Pre-Condition und Post-Condition sind bekannt und können erklärt werden.
- Bei eigenen Funktionen werden sinnvolle Pre- & Post-Conditions angegeben und überprüft.

Pre- & Post- Conditions



Pre-Conditions

- **Pre-Conditions** müssen erfüllt sein, damit eine Funktion korrekt ablaufen kann
 - Die entsprechenden Bedingungen für Funktionsparameter sollten in der Funktionsbeschreibung kommentiert werden
 - Von einer Funktion ohne kommentierte Pre-Conditions wird erwartet, dass sie mit allen möglichen Parameterwerten funktioniert
 - Wird eine Pre-Condition verletzt, führt dies per Definition zu undefiniertem Verhalten
- Wenn eine Pre-Condition verletzbar ist, hat man meist zwei Möglichkeiten:
 - 1. Die Argumente auf die Bedingung überprüfen, den Fehler auf beliebige Art mitteilen und geeignet mit der Situation umgehen (z.B. Neue Werte bei einer Konsoleneingabe einfordern)
 - 2. Verantwortung an den Aufrufer abgeben:
Die mögliche Verletzung ignorieren und davon ausgehen, dass die Funktion immer mit passenden Argumenten aufgerufen wird.
 - Es ist generell einfacher, eine geworfene Exception zu debuggen (Fall 1) als ein undefiniertes Verhalten (Fall 2).

```
void f(const std::string& s);  
// precondition: s.length() <= 32
```

Pre- & Post- Conditions



Post-Conditions

- Bedingungen für den Rückgabewert einer Funktion heißen **Post-Conditions**
 - Auch diese sollten in der Funktionsbeschreibung angegeben sein
 - z.B.: „<returns>Prozentzahl zwischen 0 und 1</returns>“
 - Auch Post-Conditions können und sollten besonders in der Entwicklungsphase vor dem Verlassen der Funktion auf dem Rückgabewert überprüft werden.

Pre- & Post- Conditions

Beispiel



```
/// <summary>
/// Berechnet die relative Häufigkeit einer Augenzahl für eine gegebene Menge an gewürfelten Augenzahlen.
/// </summary>
/// <param name="diceRolls">Nichtleere Menge an Ergebnissen des wiederholten Würfelwurfs.</param>
/// <param name="number">Augenzahl im Bereich [1,6] für die die relative Häufigkeit bestimmt werden soll.</param>
/// <returns>Relative Häufigkeit zwischen 0 und 1 der gegebenen Augenzahl oder 0 bei leerer Menge.</returns>
float calculateFrequency(const std::vector<int>& diceRolls, int number)
{
    // Überprüfe Pre-Condition auf der Nummer
    if (number < 1 || number > 6)
        throw DiceError("Wrong number of pips.");
    // Überprüfe Pre-Condition auf dem Vector
    if (diceRolls.empty()) return 0;
    int count = std::count(diceRolls.begin(), diceRolls.end(), number);
    float frequency = (float)count / diceRolls.size();
    // Überprüfe Post-Condition
    if (frequency < 0.0f || frequency > 1.0f)
        throw DiceError("Frequency not between 0 and 1");
    return frequency;
}
```

Pre- & Post- Conditions



Häufige **falsche** Gedankengänge

- „Die Funktion wird bestimmt immer mit korrekten Werten aufgerufen!“
 - Der Grund funktioniert nur, wenn man genau weiß, von wem und wann die Funktion aufgerufen wird.
 - Wenn durch einen Algorithmus bereits die Korrektheit der Werte absolut sicher gewährleistet ist, lohnt sich der Verzicht auf die Bedingungen.
 - In den meisten Fällen (besonders, wenn der Aufrufer der Funktion unbekannt ist) ist dies **kein vernünftiger Grund**, um auf die Überprüfung von Bedingungen zu verzichten.
- „Die Bedingungen verlangsamen mein Programm zu sehr!“
 - Der Grund ist **in deutlich weniger Situationen korrekt** als man denken würde und meistens ein gutes Beispiel für „vorschnelle Optimierung“.
 - Überprüfungen können jederzeit wieder entfernt werden, wenn sie sich wirklich negativ auswirken. Die Sicherheit und Korrektheit, die sie gewährleisten, lässt sich aber nicht mehr so schnell aufbauen und die Fehler, die sich mit ihnen sofort finden lassen, können einem ansonsten (ohne die Überprüfungen) schlaflose Nächte bereiten.

Pre- & Post- Conditions



Häufige **falsche** Gedankengänge

- „Die Bedingungen sind zu kompliziert zu überprüfen!“
 - Beim ersten Schreiben einer Funktion lohnt es sich immer, einfache Überprüfungen von Pre-Conditions durchzuführen.
 - Die Bedingungen in Kommentaren niederzuschreiben, kann bereits einen großen Mehrwert bringen
 - Wenn sowohl Überprüfungen als auch das Verfassen von Kommentaren zu Pre-Conditions zu kompliziert scheint, liegt das häufig daran, dass man selbst noch nicht gut genug verstanden hat, wozu man die Funktion eigentlich schreibt.

Pre- & Post- Conditions



Zusammenfassung

- **Pre-Conditions** sind Bedingungen, die **vor Ausführung einer Funktion erfüllt** sein müssen, um ihr korrektes Verhalten zu gewährleisten
- **Post-Conditions** sind Bedingungen, die **am Ende einer Funktion erfüllt** sein müssen, um einfach vermeidbare Fehler schnell ausfindig zu machen
- Pre- und Post- Conditions sollten **immer dokumentiert** werden
 - Z.B. in den **Kommentaren** der Funktionsbeschreibung
- Pre- und Post- Conditions **stets zu überprüfen** hilft beim Debuggen und bei der Gewährleistung der Funktionalität
 - Auf die Überprüfung kann verzichtet werden, wenn die Programmiersprache die Bedingung bereits erzwingt
 - z.B. kann ein `uint16_t` keine negativen Werte annehmen

Pre- & Post- Conditions

Verständnisfragen

PIN: 849822

<https://poll.hs-kempten.de/poll/>



- **Wann sollte man eine Pre-Condition in C++ normalerweise nicht testen?**
 - Wenn es kompliziert ist, sie zu überprüfen.
 - Bei Funktionen, die in einer sicheren und kontrollierten Umgebung ausgeführt werden, in der die Eingabewerte garantiert gültig sind.
 - Bei öffentlichen Funktionen, die von externen Benutzern oder anderen Teilen des Codes aufgerufen werden.
 - Wenn die Pre-Condition bereits von anderen Funktionen im Code überprüft wurde.
 - Wenn die Pre-Condition im Code ausreichend kommentiert ist.
 - Wenn die Funktion nur in seltenen Fällen fehlerhafte Eingaben erhält.
 - Wenn die Funktion zwingend schnelle Reaktionszeiten erfordert und die Überprüfung den Durchsatz beeinträchtigen würde.

Pre- & Post- Conditions

Verständnisfragen

PIN: 849822

<https://poll.hs-kempten.de/poll/>



■ Wann sollte man in C++ normalerweise keine Post-Condition testen?

- Wenn die Funktion immer korrekte Ergebnisse zurückgibt.
- Wenn die Funktion öffentlich ist und von externen Benutzern oder anderen Teilen des Codes aufgerufen wird.
- Wenn die Funktion ausschließlich von einem erfahrenen Benutzer aufgerufen wird, der die erforderlichen Nachbedingungen kennt.
- Wenn das Ergebnis ausschließlich zu Debug-Zwecken und nicht für den weiteren Programmablauf verwendet wird.
- Wenn die Verletzung der Post-Condition nicht zum Programmabsturz führt.
- Wenn sich die Post-Condition nur zu Teilen überprüfen lässt.
- Wenn die Funktion ein hohes Maß an Fehlertoleranz aufweisen muss.
- Wenn die Funktion fehlerhafte Ergebnisse nur in extrem seltenen Randfällen zurückgeben würde.

Fehlerbehandlung



Ratschläge

- Nutze Exceptions ausschließlich zur Fehlerbehandlung (für Ausnahmesituationen)
 - Wirf einen Fehler, wenn eine Funktion ihre Aufgabe nicht mehr erfüllen kann.
 - Dass sich ein Dokument nicht öffnen lässt oder das Ende einer Iteration nicht erreicht werden kann, sind erwartbare Ereignisse und keine Exceptions.
 - Nutze Exceptions, wenn sich der Fehler durch viele Aufrufe durchpropagiert.
- Nutze *noexcept*, wenn du dir sicher bist, dass eine Funktion keine Fehlerobjekte wirft und auch in Zukunft nicht werfen wird.
- Nutze Fehlercodes, wenn der direkte Aufrufer der Funktion sich um den Fehler kümmern sollte.
- Kommentiere für Funktionen ihre Pre- und Post-Conditions.
- Bevorzuge RAII gegenüber expliziten Try-Klauseln wann immer möglich.
- Alles, was zur Compile-Zeit überprüft werden kann, sollte dort überprüft werden.
 - Der Compiler ist dein bester Freund!



Ausblick: Std::Expected

Lernziele

- Dieses Kapitel soll einen Ausblick auf das von *std::expected* eingeführte Konzept zur Fehlerbehandlung geben
- Die Idee, die *std::expected* zur Fehlerbehandlung beisteuert, wurde verstanden

Fehlerbehandlung mit Std::Expected

C++17 Optional



- **std::optional** ist ein Datentyp, der einen Wert haben kann, aber nicht muss
 - Besonders praktisch bei Berechnungen, die ein Ergebnis haben können (z.B. Datenbankabfragen)
 - Mit verschiedenen Konstruktoren und der Funktion *std::make_optional* kann ein optionales Objekt mit oder ohne Wert definiert werden.
 - Über Bedingungen kann überprüft werden, ob ein Wert gesetzt ist
 - Mit *value()* lässt sich auf den Wert zugreifen, wenn das Objekt einen enthält. Enthält das Objekt keinen Wert, wird ein *std::bad_optional_access* Fehler geworfen.
 - *value_or()* gibt den gesetzten Wert, falls vorhanden, oder ansonsten einen Defaultwert zurück.

```
#include <optional>
```

```
std::optional<int> getFirst  
(const std::vector<int>& vec) {  
    if (vec.empty()) return std::nullopt;  
    else return vec[0];  
}  
  
int main() {  
    std::vector<int> myVec{1, 2, 3};  
    std::vector<int> myEmptyVec;  
    auto myInt= getFirst(myVec);  
    // Überprüfe, ob ein Wert gesetzt ist  
    if (myInt) {  
        std::cout << "*myInt: " << *myInt << '\n';  
        std::cout << "myInt.value(): "  
            << myInt.value() << '\n';  
    }  
    auto myEmptyInt= getFirst(myEmptyVec);  
    std::cout << "myEmptyInt.value_or(2017): "  
        << myEmptyInt.value_or(2017) << '\n';  
}
```

```
*myInt: 1  
myInt.value(): 1  
myEmptyInt.value_or(2017): 2017
```

Fehlerbehandlung mit Std::Expected



C++23 Optional: Monadische Operationen

- C++23 führt Operationen für `std::optional` ein, mit denen Kompositionen auf einem optionalen Objekt ermöglicht werden
 - **`and_then()`** gibt das Ergebnis eines Funktionsaufrufs oder ein leeres Objekt zurück
 - **`transform()`** transformiert den Wert des Objekts und gibt gegebenenfalls ein leeres Objekt zurück
 - **`or_else()`** ruft eine angegebene Funktion auf, falls das Objekt keinen Wert besitzt

```
#include <optional>
```

```
std::optional<int> toInt(std::string arg) {  
    try { return {std::stoi(arg)}; }  
    catch (...) { return std::nullopt; }  
}  
  
std::optional<std::string> addNumber  
    (const std::string& s, int i)  
{  
    return std::optional{s}.and_then(toInt)  
        .transform( [i](int n) { return n + i; })  
        .transform( [] (int n) { return std::to_string(n); })  
        .or_else([] { return std::optional{"NaN"s}; });  
}  
  
int main() {  
    std::cout << *addNumber("66", 100) << " "  
        << *addNumber("foo", 100) << " "  
        << *addNumber("-5", 100);  
}
```

Output:

166 NaN 95

Fehlerbehandlung mit Std::Expected

C++23 Expected



- `std::expected<T, E>` besitzt entweder einen **erwarteten Wert** vom Typ T oder einen **unerwarteten Wert** vom Typ E
 - Ermöglicht, Funktionen zu programmieren, die entweder einen Wert oder einen Fehler zurückgeben.
 - Der gespeicherte Wert wird direkt innerhalb des vom erwarteten Objekt belegten Speichers zugewiesen.
 - `std::expected` besitzt eine ähnliche Schnittstelle wie `std::optional`, nur dass `std::expected` eine Fehlermeldung zurückgeben kann
 - Wird mit `value()` auf einen Wert zugegriffen, der nicht gesetzt ist, wird ein `std::bad_expected_access` Fehler geworfen
 - Mit `std::unexpected` lässt sich der Fehlerwert setzen und mit `error()` kann auf ihn zugegriffen werden

```
#include <expected>
```

```
std::expected<int, std::string> toInt  
(const std::string& arg)  
{  
    try { return { std::stoi(arg) }; }  
    catch (...) { return std::unexpected{  
        std::string(arg + ": NaN")};  
    }  
}  
  
int main() {  
    std::vector<std::string> strings =  
        {"42", "foo"};  
    for (const auto& s: strings) {  
        auto res = toInt(s);  
        if (res) std::cout << res.value() << ' ';  
        else    std::cout << res.error() << ' ';  
    }  
}
```

Output:

```
42 foo: NaN
```